



Sandya Mannarswamy

CODE SPORT

This month's column continues the discussion on JavaScript language features. For the last couple of months, the topic of dynamic languages such as JavaScript and how they differ from traditional statically compiled languages like C/C++ has been covered. In this column, the focus is on the functional nature of JavaScript.

In languages like C or C++, functions are associated with a specific piece of work that needs to be done. So we define functions, which take inputs in the form of arguments passed to them or through global variables, then operate on their inputs, either in terms of computations or performing I/O, and then return a result value to the caller. Typically, functions are invoked using the function name directly, or through a function pointer. Passing functions as arguments is not that common, except as function pointers in C, or as functor objects in the case of C++. A typical example of a function passed as an argument to another function is C's *qsort*, which is the quick-sort implementation in the C Standard library. Here is its declaration:

```
void qsort(void *start, size_t num_elems, size_t
elem_size, int (*my_comparator)(const void *,
const void *));
```

In the above declaration, *qsort* takes four arguments; the last is a function, which does the comparison between two elements of the array to be sorted, and returns an integer, which is the result of the comparison. One would typically invoke *qsort* as below:

```
int my_comparator(const void* elem_ptr1, const
void* elem_ptr2)
{
    /* perform your comparison here */
}

int main(void)
{
    -----
    Qsort(int_array, N, sizeof(int), my_
comparator);
}
```

This is an example of what is known as a higher-order function—one which either takes a function as an argument, or returns a function as its return value. A typical use-case for passing functions in operating systems code is as 'callback functions'. This will be executed when a certain condition (such as the elapsing of a timer) is met. Functions that are not higher-order functions are known as first-order functions.

Now, you may ask why we're talking about higher-order functions. You will see these very frequently in JavaScript, so it would help to be familiar with them.